Eventra — Project Report

Eventra — Project Report

Project: Eventra — Event Registration & Ticketing Platform **Report date:** September 24, 2026 **Report scope:** Whole project — architecture, feature inventory, data layer, verification status, and open items. **Repository:** `eventflow-hub-main` (Vite + React frontend, Express/Sequelize API, MySQL)

---

1. Executive summary

Eventra is a full-stack event discovery, registration and ticketing application for the Indian market. A visitor picks a **state and city**, sees only events in that location, filters and sorts them, registers for a seat, and pays through a mock Razorpay checkout that issues a printable e-ticket. An admin area exposes users, events, registrations and payments with search, CSV export and bulk delete.

The project is **feature-complete for a demo/portfolio deployment** and runs in two modes:

Mode	Requirements	What it demonstrates
Static demo (`preview.html`)	None — no Node, no DB	The entire app with 313 generated events served by an in-page `fetch` shim
Real stack (`npm run dev:all`)	Node + MySQL	The same app against Sequelize models and the Express API

As of this report, the **static demo runs live** and every code-level check passes: TypeScript 0 errors, unit tests green, production build succeeds, and the location → events flow was driven end-to-end in a real browser.

---

2. Project overview

Problem it solves. Generic event listings show everything at once, which is useless when events are geographically specific. Eventra inverts the flow: location is chosen **before** any events render, so every listing the user sees is relevant by construction.

Primary user journeys

1. **Discover** — pick state → city → browse events in that city, filter by category, sort by date, search by title or venue.
2. **Register** — 3-step stepper (details → ticket type → review) with live seat-availability feedback.
3. **Pay** — mock Razorpay checkout, animated success screen, printable e-ticket with a barcode strip.
4. **Administer** — CRUD events (with State/City pickers), inspect users/registrations/payments, export CSV, bulk delete.

---

3. Technology stack

Layer	Technology
UI framework	React 18 + TypeScript, Vite
Routing	React Router (hash-based in static builds)
Styling	Tailwind CSS + shadcn/ui primitives (60 components), custom design tokens
Animation	Framer Motion, CSS 3D keyframes (intro corridor, jelly hero)
Data fetching / cache	TanStack React Query
Charts	Recharts
Forms & validation	React Hook Form + Zod
API	Node.js + Express 4
ORM / database	Sequelize → MySQL
Tests	Vitest + @testing-library
Build tooling	Vite (bundle), pandoc + headless Chrome (this report), custom preview builders

Code size (excluding the shadcn/ui primitive library):

Area	Lines
Pages (`src/pages` , 9 pages)	3,010
App components (`src/components`)	1,532
lib / hooks / utils	445
Server (`server/` , 16 files)	983
Build & data scripts (`scripts/`)	769
Tests	62
Total	**~7,100**
UI primitives (`src/components/ui` , vendored)	4,596

---

4. Architecture

4.1 Frontend

```
src/
  pages/         Dashboard, Events, Register, Payment, RazorpayCheckout,
                 Admin, Login, Signup, NotFound
  components/     Layout, LocationGate, IntroScreen, Footer, UserMenu,
                  NotificationCenter, AnimatedNumber, Breadcrumbs,
                  EmptyState, PageTransition, Reveal, Skeletons
  components/ui/  shadcn/ui primitives + hero templates
  lib/            api.ts (typed API client), location.ts (states/cities +
                  geolocation), utils.ts
  hooks/          usePageTitle
```

- **`src/lib/api.ts`** is the single typed gateway to the API (`Event`, `User`, `Registration`, `Payment`, `Stats` types plus `fetchEvents/fetchEvent/createEvent/...`).
- **`src/components/LocationGate.tsx`** owns the blocking location dialog (state → city → *Show Events*, "Use my current location", loading and error states).
- **`src/lib/location.ts`** fetches states/cities from the keyless `countriesnow.space` API with a 24-hour `localStorage` cache, falls back to the bundled `public/data/states-cities.json`, and reverse-geocodes coordinates via OpenStreetMap Nominatim.

4.2 Backend

```
server/
  index.js              Express app, CORS, route mounting, boot-time seed
  config/database.js    Sequelize connection (.env credentials)
  models/               User, Event, Ticket, Registration, Payment (+ index)
  routes/               auth, events, users, registrations, payments, stats
  middleware/           errorHandler (asyncHandler + central error response)
  seedLocations.js      Auto-migration + idempotent event seeding
```

4.3 Data model

Table	Key columns	Notes
`user`	`user_id`, name, email, password, role	auth + admin flag
`event`	`event_id`, title, venue, **state**, **city**, description, category, capacity, status, organizer_id, start_date, end_date	`state` / `city` added for the location flow; auto-migrated on boot
`ticket`	composite PK (`ticket_id`, `event_id`), type, price, quantity_available	one “Regular” ticket per seeded event
`registration`	registration id, user, event, ticket type, status	Confirmed / Pending
`payment`	payment id, user, amount, method, status	UPI / Card / Razorpay

4.4 API surface

Method	Endpoint	Purpose
POST	`/api/auth/login`, `/api/auth/signup`	Authentication
GET	`/api/events`	List events — filters `?state=&city=&category=` (case-insensitive)
GET	`/api/events/locations`	Distinct state → cities that actually have events
GET	`/api/events/:id`	Single event
POST / PUT / DELETE	`/api/events`, `/api/events/:id`	Event CRUD (accepts `state`, `city`)
GET	`/api/users`, `/api/users/:id`	Users
GET / POST	`/api/registrations`	Registrations
GET / POST	`/api/payments`	Payments
GET	`/api/stats`, `/api/stats/registrations-per-event`	Dashboard metrics

Unknown `/api/*` paths return a JSON 404 rather than falling through to the SPA.

---

5. Location-first flow (core differentiator)

1. Opening `/events` with no saved location shows a **blocking "Select your location"** dialog.
2. **Step 1 — State:** a dropdown of all 32 states/UTs.
3. **Step 2 — City:** enabled only after a state is chosen; shows a "Loading cities…" indicator while the list resolves.
4. **Step 3 — Show Events:** persists the choice to `localStorage` (`eventra-location`) and renders only events matching that **state AND city** (case-insensitive, filtered server-side).
5. The page header reads **"Events in Chennai, Tamil Nadu"** with a **Change Location** button that clears the saved value and reopens the dialog at step 1.
6. Returning visitors skip the dialog entirely; a city with no events shows a friendly empty state plus clickable shortcuts to sibling cities in the same state that *do* have events.
7. **Optional bonus:** "Use my current location" uses the browser Geolocation API plus Nominatim reverse geocoding, with alias mapping (Bangalore → Bengaluru); on denial or failure it falls back to the manual dropdowns.

---

6. Demo dataset (generated)

The dataset is produced by a seeded generator so the output is byte-identical on every run, and lives in **one clearly named file** (`public/data/events.json`) so it can be swapped for real database content later.

```
npm run events:generate   # scripts/generateEvents.cjs  → public/data/events.json
npm run events:validate   # scripts/validateEvents.cjs  → coverage report
```

6.1 Coverage

Metric	Value
Events	**313**
Cities covered	**157 / 157** (100%)
States / UTs	32
Tier-1 metro events	120
Date range	2026-09-30 → 2027-01-23 (~4 months)
Free events	71
Paid events	242
Events per city	min 1, max 7

6.2 Tier-1 metros (5–7 events each)

City	Events	City	Events	City	Events	City	Events
Mumbai	6	New Delhi	5	Bengaluru	6	Hyderabad	7
Chennai	6	Kolkata	5	Pune	6	Ahmedabad	7
Surat	7	Jaipur	5	Lucknow	7	Kanpur	5
Nagpur	5	Indore	5	Bhopal	5	Visakhapatnam	6
Patna	7	Vadodara	7	Coimbatore	6	Kochi	7

Every Tier-1 city carries **at least 4 distinct categories**; every other city in the dropdown gets **1–2 events**, so no city can render an empty list.

6.3 Category spread

Category	Count	Category	Count	Category	Count
Music	36	Art and Culture	35	Business and Networking	33
Fitness	31	Charity	29	Education	27
Workshop	27	Food Festival	26	Sports	25
Tech and Startup	24	Comedy	20		

6.4 Realism rules

- **Venue names scale with the city** — metros get convention centres, stadiums and arenas; smaller cities get town halls, college grounds and community halls.
- **Titles are unique** — each is city-flavoured ("Sufi Nights: Chennai", "Erode Theatre Night") with a dedupe pass guaranteeing no repeats.
- **Dates mix weekdays and weekends** and always fall in the future relative to the generation anchor.
- **Prices tier by city size** — free entry for ~25% of events, ₹50–₹500 in smaller cities, up to ₹1,500 in metros.
- **Every event carries** `id, title, category, description, date, time, venue, city, state, price, totalSeats, seats, image`.

6.5 Validation rules enforced

`scripts/validateEvents.cjs` fails the build (exit code 1) if any of these break:

1. any city in `states-cities.json` has zero events;
2. any Tier-1 metro falls outside 5–7 events, or has fewer than 4 categories;
3. any event id is duplicated;
4. any required field is missing, or `seats > totalSeats`;
5. any event date is not in the future.

Latest run: **all checks passed** — 313 events, 157 cities, 120 Tier-1 events, 32 states, 20 Tier-1 metros, 0 warnings.

7. Build & tooling pipeline

Script	Purpose
`scripts/generateEvents.cjs`	Seeded generator (mulberry32, seed `20260924`) reading `states-cities.json`
`scripts/validateEvents.cjs`	Coverage / tier / id / date validation with a printed summary
`scripts/build-preview.cjs`	Builds the self-contained `preview.html`: inlines the production bundle, CSS, images and a stateful mock API shim
`scripts/serve-preview.cjs`	Zero-dependency static server for the demo (port 3000, no-cache headers, `/api/health`)
`scripts/report.css`	Print stylesheet for PDF reports (this document)

The preview shim serves `/api/events` with `state` / `city` / `category` filtering, `/api/events/locations`, the states-cities fallback and stateful payment/registration mutations, so the demo behaves like the real backend without a database.

8. Verification status

Check	Command	Result
TypeScript	`tsc --noEmit -p tsconfig.app.json`	✅ 0 errors
Unit tests	`vitest run`	✅ 4/4 passed (`src/lib/location.test.ts`)
Production build	`vite build`	✅ succeeds
Dataset validation	`node scripts/validateEvents.cjs`	✅ all checks passed
Live browser — Tier-1 city	`/events` → Chennai, Tamil Nadu	✅ "6 events in Chennai, Tamil Nadu"
Live browser — Tier-2 city	`/events` → Erode, Tamil Nadu	✅ "1 event in Erode, Tamil Nadu"
Live browser — category filter	"Music" pill in Chennai	✅ narrows 6 → 1 event
Live browser — date sort	soonest / latest toggle	✅ 2 Oct 2026 → 11 Jan 2027, reverses correctly
Live browser — empty city	Coimbatore before seeding	✅ friendly message + sibling-city shortcuts
Dashboard wiring	`/` stat cards	✅ "313 events live"

Defects found and fixed during this cycle

1. `events.json` category filtering was missing from the preview shim (returned all city events) — added.
2. Generated images (`/events/*.png`) failed to resolve through React's `img.src` assignment — added a property-setter hook alongside the `setAttribute` hook.
3. The `fetchEvents` signature change broke React Query type inference in three pages (`Admin`, `Index`, `Register`) — wrapped call sites in `() => fetchEvents()`.
4. The MySQL seeder created tickets without the composite primary key — restored explicit `ticket_id` / `event_id`.

---

9. How to run

Demo (no database)

```
# regenerate + verify data (optional)
npm run events:generate
npm run events:validate

# build and serve the static demo
node scripts/build-preview.cjs
node scripts/serve-preview.cjs        # http://localhost:3000
```

Full stack (requires MySQL)

```
npm run dev:all        # Vite on :3000 + Express API on :5000
```

On first boot the server auto-migrates the `event` table (adds `state` / `city`) and idempotently seeds all 313 events with tickets.

---

10. Known limitations & environment notes

Item	Detail	Impact
MySQL credentials	The `.env` password is rejected (`Access denied for 'root'`), so the DB-backed stack cannot boot as configured.	Blocking for the real-stack mode; fix `DB_PASS` to resolve.
Port 5000	macOS AirPlay/ControlCenter occupies port 5000 on this machine.	Run the API on 5001: `PORT=5001 API_PORT=5001 npm run dev:all` (the Vite proxy reads `API_PROXY_TARGET`).
Node not on PATH	No system Node/npm/bun; builds were verified with a Node 22 binary in `/tmp`.	Procedures documented in `.freebuff/run.md`; run `npm install` locally to restore native binaries.
Synthetic telemetry	Seeded `seats`, prices and demo payments are generated data, and the dashboard charts derive from the demo/shim payment rows.	Expected for demo data; replace via the admin panel or real registrations.
External lookups	States/cities API and Nominatim geocoding are free and keyless; a slow/offline network falls back to the bundled JSON.	Degrades gracefully; swap to a keyed provider via `.env` if needed.

---

11. Suggested next steps

1. **Fix** `DB_PASS` and seed MySQL so the demo data and the database agree.
2. **Replace synthetic data with real events** — the seeder reads `events.json`, so swapping that one file (or pointing it at a live query) is the whole migration.
3. **Deploy the static demo** (GitHub Pages/Vercel) so it is shareable without a local server.
4. **Add dataset regression tests** so the per-city coverage rules can't silently break.
5. **Location-aware dashboard** — highlight "events near you" and support saved favourite cities.

Report generated from a live inspection of the working tree: dataset statistics, validation output, typecheck, test run and browser verification on September 24, 2026.